

Relatório de Testes

HOSPED — Microserviço de Reservas e Hóspedes

Versão 1.0

Documento baseado no padrão do relatório de testes do MS-Pagamentos, adaptado para os requisitos funcionais, requisitos não funcionais, regras de negócio e casos de uso CDU0016 a CDU0031.

Versão	Data	Alterações	Fonte	Autoria
1.0	04/05/2026	Versão inicial do documento de testes planejados	CDU0016 a CDU0031; requisitos e regras informados	Grupo HOSPED

SUMÁRIO

Seção	Descrição
1	Introdução
2	Sistema testado
3	Requisitos testados
3.1	Requisitos funcionais
3.2	Requisitos não funcionais
3.3	Regras de negócio
4	Matriz de rastreabilidade
5	Especificação dos casos de teste planejados
5.1-5.29	Reservas, check-in/check-out, comentários e mensageria
5.30-5.43	Hóspedes
5.44-5.48	Requisitos não funcionais transversais
6	Resumo da classificação dos resultados
7	Conclusão

1. INTRODUÇÃO

Este documento constitui o Relatório de Testes planejados do microserviço de reservas e hóspedes do sistema HOSPED, desenvolvido no contexto de uma aplicação de gerenciamento hoteleiro baseada em arquitetura de microserviços.

O objetivo é registrar a rastreabilidade entre requisitos funcionais, requisitos não funcionais, regras de negócio e casos de uso, além de descrever os casos de teste necessários para validar os fluxos de criação de reservas, consulta, check-in, check-out, confirmação de pagamento, processamento de eventos externos, comentários de encerramento e manutenção cadastral de hóspedes.

Como não foram fornecidas evidências de execução real dos testes, os casos estão classificados como Planejados, com o campo Resultado Obtido definido como “A executar”. Após a implementação e execução automatizada/manual, esta seção poderá ser atualizada com evidências reais, status de validação e bugs encontrados.

2. SISTEMA TESTADO

O sistema HOSPED é uma aplicação distribuída para gestão hoteleira. O escopo deste relatório considera o microserviço responsável por reservas e hóspedes, incluindo endpoints REST, regras de domínio, persistência, histórico de alterações, publicação de eventos de domínio e consumo de eventos de pagamento via mensageria.

As funcionalidades analisadas correspondem aos casos de uso CDU0016 a CDU0031: criar e consultar reservas, listar reservas por hóspede, realizar check-in e check-out, confirmar pagamento antecipado de forma manual ou por evento externo, cancelar reservas por pagamento expirado, registrar comentário de encerramento e manter o cadastro de hóspedes.

3. REQUISITOS TESTADOS

A tabela de priorização utiliza Impacto x Probabilidade para indicar criticidade. Valores mais altos representam maior prioridade de cobertura nos testes.

3.1. Requisitos funcionais

ID	Descrição do Requisito Funcional	Impacto	Prob.	Prioridade
RF013	O sistema deve permitir criar reserva vinculada a hóspede e quarto, com pagamento no hotel ou antecipado.	9	9	81
RF014	O sistema deve permitir listar todas as reservas cadastradas.	5	5	25
RF015	O sistema deve permitir buscar reserva por identificador único.	7	5	35
RF016	O sistema deve permitir listar o histórico de reservas de um hóspede.	7	5	35
RF017	O sistema deve permitir realizar check-in de reservas elegíveis.	9	9	81
RF018	O sistema deve permitir realizar check-out de reservas ativas.	9	9	81
RF019	O sistema deve permitir confirmar manualmente pagamento antecipado de reserva elegível.	9	7	63
RF020	O sistema deve permitir registrar comentário de encerramento em reserva encerrada.	5	5	25
RF021	O sistema deve confirmar pagamento antecipado a partir de evento externo aprovado.	9	7	63
RF022	O sistema deve cancelar reserva antecipada a partir de evento externo de pagamento expirado.	9	7	63
RF023	O sistema deve permitir cadastrar hóspedes com CPF válido e único.	9	9	81
RF024	O sistema deve permitir listar hóspedes cadastrados.	5	5	25
RF025	O sistema deve permitir buscar hóspedes por ID e por CPF.	7	5	35
RF026	O sistema deve permitir atualizar dados cadastrais e status ativo de hóspedes.	7	7	49
RF027	O sistema deve permitir excluir hóspede sem reservas vinculadas.	7	7	49
RF028	O sistema deve integrar-se ao serviço de quartos para disponibilidade e check-out quando aplicável.	9	7	63
RF029	O sistema deve registrar histórico e publicar eventos de domínio nas alterações relevantes de reserva.	9	7	63

3.2. Requisitos não funcionais

ID	Descrição do Requisito Não Funcional	Impacto	Prob.	Prioridade
RNF001	As APIs HTTP devem retornar códigos e mensagens de erro padronizados para validação, regra de negócio, conflito, integração e recurso não encontrado.	9	7	63
RNF002	Falhas em consumidores assíncronos e integrações externas devem ser registradas em log com contexto suficiente para diagnóstico.	7	7	49
RNF003	Eventos assíncronos inválidos, duplicados ou incompatíveis devem ser tratados sem interromper o serviço.	9	7	63
RNF004	As funcionalidades devem ser acessíveis somente aos atores autorizados: administrador e funcionário, quando houver camada de segurança habilitada.	9	5	45
RNF005	Listagens sem registros devem retornar lista vazia, sem erro HTTP.	5	7	35
RNF006	Integrações externas devem ser tolerantes a falhas, evitando inconsistência da reserva quando a falha não for bloqueante.	9	7	63
RNF007	O CPF deve ser validado, normalizado e persistido de forma consistente para impedir duplicidade lógica.	9	9	81
RNF008	Operações de mudança de estado de reserva devem manter rastreabilidade por histórico.	9	7	63

3.3. Regras de negócio

ID	Descrição da Regra de Negócio	Impacto	Prob.	Prioridade
RN012	Toda reserva deve estar vinculada a um hóspede existente.	9	9	81
RN013	Para criar reserva, deve ser informado quartoId ou quartoServicold.	9	9	81
RN014	A data de check-out deve ser posterior à data de check-in.	9	9	81
RN015	Quando houver quartoServicold, a disponibilidade externa deve ser consultada.	9	7	63
RN016	Toda reserva criada deve iniciar com status PENDENTE e histórico registrado.	9	9	81
RN017	Reserva com pagamento antecipado exige valorDiaria e metodoPagamento e deve publicar evento para criação do pagamento.	9	9	81
RN018	Reserva com pagamento no hotel deve seguir o fluxo PENDENTE -> ATIVA -> ENCERRADA.	9	7	63
RN019	Reserva com pagamento antecipado deve seguir o fluxo PENDENTE -> PAGA -> ATIVA -> ENCERRADA.	9	7	63
RN020	Check-in de reserva antecipada só pode ocorrer quando o pagamento estiver confirmado.	9	9	81
RN021	Check-out só pode ocorrer para reserva ATIVA; falha não bloqueante no serviço de quartos deve ser apenas registrada.	9	7	63
RN022	Comentário de encerramento só pode ser registrado em reserva ENCERRADA e deve ter de 1 a 1000 caracteres.	7	7	49
RN023	Evento de pagamento APROVADO confirma somente reserva PAGO_ANTECIPADO elegível.	9	7	63
RN024	Evento de pagamento EXPIRADO cancela somente reserva PAGO_ANTECIPADO elegível.	9	7	63
RN025	Eventos para reservas inexistentes, não antecipadas ou em status final devem ser ignorados ou logados sem quebra do consumer.	9	7	63
RN026	CPF de hóspede deve ser válido, normalizado e único.	9	9	81
RN027	Novo hóspede deve ser cadastrado como ativo.	7	7	49
RN028	Atualização de hóspede exige existência do registro e CPF não pertencente a outro hóspede.	9	7	63
RN029	Hóspede com reservas vinculadas não pode ser excluído.	9	9	81
RN030	Consultas de listagem sem registros devem retornar lista vazia.	5	7	35

4. MATRIZ DE RASTREABILIDADE

CDU	Funcionalidade	RF	RN	RNF	Casos de Teste
CDU0016	Criar reserva	RF013, RF019, RF028, RF029	RN012-RN017	RNF001, RNF006, RNF008	TC-RES-001 a TC-RES-008
CDU0017	Listar reservas	RF014	RN030	RNF005	TC-RES-009 a TC-RES-010
CDU0018	Buscar reserva por ID	RF015	RN030	RNF001	TC-RES-011 a TC-RES-012
CDU0019	Listar reservas por hóspede	RF016	RN030	RNF005	TC-RES-013 a TC-RES-014
CDU0020	Realizar check-in	RF017, RF019, RF029	RN018-RN020	RNF001, RNF008	TC-RES-015 a TC-RES-018
CDU0021	Realizar check-out	RF018, RF028, RF029	RN021	RNF002, RNF006, RNF008	TC-RES-019 a TC-RES-021
CDU0022	Confirmar pagamento antecipado	RF019, RF029	RN019, RN020, RN025	RNF001, RNF008	TC-RES-022 a TC-RES-023
CDU0023	Registrar comentário de encerramento	RF020	RN022	RNF001	TC-RES-024 a TC-RES-025
CDU0024	Confirmar pagamento por evento externo	RF021, RF029	RN023, RN025	RNF002, RNF003	TC-RES-026 a TC-RES-027
CDU0025	Cancelar reserva por pagamento expirado	RF022, RF029	RN024, RN025	RNF002, RNF003	TC-RES-028 a TC-RES-029
CDU0026	Cadastrar hóspede	RF023	RN026, RN027	RNF001, RNF007	TC-HOSP-030 a TC-HOSP-033
CDU0027	Listar hóspedes	RF024	RN030	RNF005	TC-HOSP-034 a TC-HOSP-035
CDU0028	Buscar hóspede por ID	RF025	RN030	RNF001	TC-HOSP-036 a TC-HOSP-037
CDU0029	Buscar hóspede por CPF	RF025	RN026	RNF001, RNF007	TC-HOSP-038 a TC-HOSP-039
CDU0030	Atualizar hóspede	RF026	RN026, RN028	RNF001, RNF007	TC-HOSP-040 a TC-HOSP-041
CDU0031	Excluir hóspede	RF027	RN029	RNF001	TC-HOSP-042 a TC-HOSP-043
Transversal	Segurança, erros, logs, desempenho e auditoria	RF029	RN012-RN030	RNF001-RNF008	TC-NF-044 a TC-NF-048

5. ESPECIFICAÇÃO DOS CASOS DE TESTE PLANEJADOS

Esta seção apresenta 48 casos de teste planejados. Eles foram estruturados seguindo o padrão do relatório de referência, com objetivo, localização, pré-condições, passo a passo, técnica, critério de seleção, entradas e resultado esperado.

Legenda de classificação: Planejado = caso especificado, mas ainda não executado; Validado = caso executado com sucesso; Bug = falha encontrada; Melhoria = ajuste desejável ou necessário identificado.

5.1. Criar reserva com pagamento no hotel

Requisito	RF013 / RF029
Regra de Negócio	RN012 / RN013 / RN014 / RN016 / RN018
Caso de Uso	CDU0016 — Criar reserva Fluxo Básico
Roteiro de Teste	RT-RES-001 — Criação de reserva presencial
Objetivo do Roteiro	Verificar se uma reserva válida com pagamento no hotel é criada com status PENDENTE, histórico registrado e evento de reserva criada publicado.
Localização	ReservaService.criarReserva(CriarReservaRequest) e POST /api/v1/reservas
Pré-condições	Hóspede existente; quartold informado; datas válidas; pagamentoModo=PAGAR_NO_HOTEL.
Passo a Passo	1. Montar request válido. 2. Mockar consulta do hóspede existente. 3. Chamar criação da reserva. 4. Verificar status, dados retornados, histórico e publicação do evento de reserva criada.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Branca e Caixa Preta
Critério de Seleção	Fluxo básico do CDU0016 para modalidade de pagamento no hotel.
Caso de Teste	TC-RES-001 — Criar reserva com pagamento no hotel
Entradas	hospedeld válido; quartold válido; checkInData=2026-06-10; checkOutData=2026-06-13; pagamentoModo=PAGAR_NO_HOTEL.
Resultado Esperado	ReservaResponse retornada com status PENDENTE; histórico criado; evento reserva.criada publicado; sem evento de pagamento antecipado.
Prioridade	Alta
Pós-condições	Reserva persistida em estado PENDENTE.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre o caminho principal de criação presencial e garante rastreabilidade inicial da reserva.

5.2. Criar reserva com pagamento antecipado

Requisito	RF013 / RF019 / RF029
Regra de Negócio	RN012 / RN013 / RN014 / RN016 / RN017 / RN019
Caso de Uso	CDU0016 — Criar reserva Fluxo Básico passos 4 e 5
Roteiro de Teste	RT-RES-002 — Criação de reserva antecipada
Objetivo do Roteiro	Verificar se reserva antecipada válida publica também evento para criação de pagamento.
Localização	ReservaService.criarReserva(CriarReservaRequest)
Pré-condições	Hóspede existente; quartold ou quartoServicold informado; valorDiaria e metodoPagamento válidos.
Passo a Passo	1. Montar request com pagamentoModo=PAGO_ANTECIPADO. 2. Chamar criação. 3. Verificar status PENDENTE. 4. Verificar histórico. 5. Verificar evento de reserva criada e evento para pagamento.
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle da modalidade PAGO_ANTECIPADO.
Caso de Teste	TC-RES-002 — Criar reserva com pagamento antecipado
Entradas	valorDiaria=300.00; metodoPagamento=PIX; datas válidas.
Resultado Esperado	Reserva criada com status PENDENTE; pagamentoStatus pendente; evento de criação de pagamento publicado.
Prioridade	Alta
Pós-condições	Reserva antecipada aguardando confirmação de pagamento.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre a principal diferença entre pagamento no hotel e pagamento antecipado.

5.3. Criar reserva — hóspede inexistente

Requisito	RF013
Regra de Negócio	RN012
Caso de Uso	CDU0016 — FE01
Roteiro de Teste	RT-RES-003 — Validação de hóspede obrigatório existente
Objetivo do Roteiro	Verificar se o sistema bloqueia reserva para hóspede inexistente.
Localização	ReservaService.criarReserva(CriarReservaRequest)
Pré-condições	Repositório/cliente de hóspede retorna inexistente para o hospedeld informado.
Passo a Passo	1. Montar request com hospedeld inexistente. 2. Chamar criação. 3. Verificar exceção de recurso não encontrado. 4. Verificar que a reserva não foi salva.
Nível de Teste	Unitário
Técnica de Teste	Partição de equivalência
Critério de Seleção	Entrada com identificador de hóspede inválido.
Caso de Teste	TC-RES-003 — Criar reserva — hóspede inexistente
Entradas	hospedeld=inexistente.
Resultado Esperado	Erro de recurso não encontrado; nenhum histórico ou evento publicado.
Prioridade	Alta
Pós-condições	Nenhuma reserva criada.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante que a integridade da reserva não seja violada sem hóspede válido.

5.4. Criar reserva — ausência de quartold e quartoServicold

Requisito	RF013
Regra de Negócio	RN013
Caso de Uso	CDU0016 — FE02
Roteiro de Teste	RT-RES-004 — Obrigatoriedade de referência de quarto

Objetivo do Roteiro	Verificar se a criação é bloqueada quando nenhum identificador de quarto é informado.
Localização	ReservaService.criarReserva(CriarReservaRequest)
Pré-condições	Request sem quartold e sem quartoServicold.
Passo a Passo	1. Montar request sem identificadores de quarto. 2. Chamar criação. 3. Verificar erro de regra de negócio. 4. Verificar ausência de persistência.
Nível de Teste	Unitário
Técnica de Teste	Partição de equivalência
Critério de Seleção	Valores ausentes para campos alternativos obrigatórios.
Caso de Teste	TC-RES-004 — Criar reserva — ausência de quartold e quartoServicold
Entradas	quartold=null; quartoServicold=null.
Resultado Esperado	Erro de regra de negócio informando que quartold ou quartoServicold deve ser informado.
Prioridade	Alta
Pós-condições	Reserva não criada.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre a validação estrutural obrigatória para vincular reserva a um quarto.

5.5. Criar reserva — datas inválidas

Requisito	RF013
Regra de Negócio	RN014
Caso de Uso	CDU0016 — FE03
Roteiro de Teste	RT-RES-005 — Validação de intervalo de hospedagem
Objetivo do Roteiro	Verificar se checkOutData deve ser posterior a checkInData.
Localização	ReservaService.criarReserva(CriarReservaRequest)
Pré-condições	Request com checkOutData igual ou anterior a checkInData.
Passo a Passo	1. Montar cenários com checkOutData igual ao checkInData e anterior ao checkInData. 2. Chamar criação. 3. Verificar erro de regra de negócio.
Nível de Teste	Unitário
Técnica de Teste	Análise de valor limite
Critério de Seleção	Fronteira entre data válida e inválida.
Caso de Teste	TC-RES-005 — Criar reserva — datas inválidas
Entradas	checkInData=2026-06-10; checkOutData=2026-06-10 ou 2026-06-09.
Resultado Esperado	Erro de regra de negócio; reserva não salva.
Prioridade	Alta
Pós-condições	Nenhuma alteração persistida.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante que a reserva sempre represente um período válido de hospedagem.

5.6. Criar reserva — quarto indisponível na integração externa

Requisito	RF013 / RF028
Regra de Negócio	RN015
Caso de Uso	CDU0016 — FE04
Roteiro de Teste	RT-RES-006 — Indisponibilidade externa de quarto
Objetivo do Roteiro	Verificar se a indisponibilidade retornada pelo serviço externo impede a criação da reserva.
Localização	ReservaService.criarReserva(CriarReservaRequest) com QuartoIntegrationClient
Pré-condições	quartoServicold informado; serviço de quartos retorna indisponível.
Passo a Passo	1. Mockar integração externa para indisponível. 2. Chamar criação. 3. Verificar erro de integração. 4. Verificar que não houve persistência.
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Caminho de integração externa do CDU0016.
Caso de Teste	TC-RES-006 — Criar reserva — quarto indisponível na integração externa
Entradas	quartoServicold=Q-101; disponibilidade=false.
Resultado Esperado	Erro de integração ou indisponibilidade; reserva não criada.
Prioridade	Alta
Pós-condições	Nenhuma reserva persistida.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Evita overbooking quando a disponibilidade depende do serviço de quartos.

5.7. Criar reserva antecipada — dados de pagamento ausentes

Requisito	RF013 / RF019
Regra de Negócio	RN017
Caso de Uso	CDU0016 — FE05
Roteiro de Teste	RT-RES-007 — Dados obrigatórios de pagamento antecipado
Objetivo do Roteiro	Verificar se pagamento antecipado exige valorDiaria e metodoPagamento.
Localização	ReservaService.criarReserva(CriarReservaRequest)
Pré-condições	Request com pagamentoModo=PAGO_ANTECIPADO sem valorDiaria ou sem metodoPagamento.
Passo a Passo	1. Montar request antecipado sem valorDiaria. 2. Montar request antecipado sem metodoPagamento. 3. Chamar criação. 4. Verificar erro de regra de negócio nos dois casos.
Nível de Teste	Unitário
Técnica de Teste	Partição de equivalência
Critério de Seleção	Campos obrigatórios condicionais à modalidade de pagamento.
Caso de Teste	TC-RES-007 — Criar reserva antecipada — dados de pagamento ausentes
Entradas	pagamentoModo=PAGO_ANTECIPADO; valorDiaria=null ou metodoPagamento=null.

Resultado Esperado	Erro de regra de negócio; nenhum evento de pagamento publicado.
Prioridade	Alta
Pós-condições	Reserva não criada.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante consistência do fluxo financeiro antecipado.

5.8. Criar reserva — dados obrigatórios inválidos

Requisito	RF013 / RNF001
Regra de Negócio	RN012 / RN014
Caso de Uso	CDU0016 — FE06
Roteiro de Teste	RT-RES-008 — Validação Bean Validation do request
Objetivo do Roteiro	Verificar se campos obrigatórios inválidos retornam erro de validação padronizado.
Localização	ReservaController POST /api/v1/reservas
Pré-condições	Controller configurado com validações do request.
Passo a Passo	1. Enviar JSON sem campos obrigatórios. 2. Enviar formatos inválidos quando aplicável. 3. Verificar HTTP 400 e corpo de erro.
Nível de Teste	Integração HTTP
Técnica de Teste	Caixa Preta
Critério de Seleção	Validação de contrato de entrada.
Caso de Teste	TC-RES-008 — Criar reserva — dados obrigatórios inválidos
Entradas	JSON com hospedeld ausente, datas ausentes ou tipos inválidos.
Resultado Esperado	HTTP 400 Bad Request com mensagem de validação; service não chamado.
Prioridade	Alta
Pós-condições	Nenhuma reserva criada.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre validações de borda do endpoint público.

5.9. Listar reservas — com registros

Requisito	RF014 / RNF005
Regra de Negócio	RN030
Caso de Uso	CDU0017 — Fluxo Básico
Roteiro de Teste	RT-RES-009 — Listagem de reservas cadastradas
Objetivo do Roteiro	Verificar se o endpoint retorna todas as reservas convertidas para DTO.
Localização	ReservaService.listarReservas() e GET /api/v1/reservas
Pré-condições	Repositório contém duas reservas.
Passo a Passo	1. Mockar findAll() com duas reservas. 2. Chamar listagem. 3. Verificar quantidade, campos principais e status HTTP 200.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Preta
Critério de Seleção	Fluxo básico de consulta.
Caso de Teste	TC-RES-009 — Listar reservas — com registros
Entradas	Sem parâmetros.
Resultado Esperado	Lista de ReservaResponse com tamanho 2.
Prioridade	Média
Pós-condições	Lista retornada ao usuário.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante que o endpoint de consulta geral funciona para dados existentes.

5.10. Listar reservas — lista vazia

Requisito	RF014 / RNF005
Regra de Negócio	RN030
Caso de Uso	CDU0017 — FE01
Roteiro de Teste	RT-RES-010 — Listagem sem reservas
Objetivo do Roteiro	Verificar se ausência de reservas retorna lista vazia sem erro.
Localização	ReservaService.listarReservas() e GET /api/v1/reservas
Pré-condições	Repositório retorna lista vazia.
Passo a Passo	1. Mockar findAll() vazio. 2. Chamar listagem. 3. Verificar HTTP 200 e array vazio.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Partição de equivalência
Critério de Seleção	Consulta com e sem dados.
Caso de Teste	TC-RES-010 — Listar reservas — lista vazia
Entradas	Sem parâmetros.
Resultado Esperado	HTTP 200 com [].
Prioridade	Baixa
Pós-condições	Nenhum dado alterado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre a regra de listagem vazia sem exceção.

5.11. Buscar reserva por ID — sucesso

Requisito	RF015
Regra de Negócio	RN030
Caso de Uso	CDU0018 — Fluxo Básico
Roteiro de Teste	RT-RES-011 — Busca de reserva por identificador
Objetivo do Roteiro	Verificar se uma reserva existente é retornada corretamente pelo ID.
Localização	ReservaService.buscarPorId(id) e GET /api/v1/reservas/{id}
Pré-condições	Reserva existente no repositório.
Passo a Passo	1. Mockar findById(id). 2. Chamar busca. 3. Verificar DTO retornado. 4. Verificar HTTP 200 no controller.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Preta
Critério de Seleção	Fluxo básico de busca por ID.
Caso de Teste	TC-RES-011 — Buscar reserva por ID — sucesso
Entradas	idReserva=res-001.
Resultado Esperado	ReservaResponse com id res-001 e dados correspondentes.
Prioridade	Média
Pós-condições	Reserva retornada.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Valida o contrato de consulta individual.

5.12. Buscar reserva por ID — não encontrada

Requisito	RF015 / RNF001
Regra de Negócio	RN030
Caso de Uso	CDU0018 — FE01
Roteiro de Teste	RT-RES-012 — Reserva inexistente
Objetivo do Roteiro	Verificar se ID inexistente retorna erro de recurso não encontrado.
Localização	ReservaService.buscarPorId(id)
Pré-condições	Repository retorna Optional.empty().
Passo a Passo	1. Chamar busca com ID inexistente. 2. Verificar exceção/HTTP 404. 3. Verificar mensagem de erro.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Partição de equivalência
Critério de Seleção	ID existente e inexistente.
Caso de Teste	TC-RES-012 — Buscar reserva por ID — não encontrada
Entradas	idReserva=inexistente.
Resultado Esperado	Erro 404 ou exceção equivalente de recurso não encontrado.
Prioridade	Média
Pós-condições	Nenhum dado alterado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante tratamento correto de busca sem resultado.

5.13. Listar reservas por hóspede

Requisito	RF016
Regra de Negócio	RN030
Caso de Uso	CDU0019 — Fluxo Básico
Roteiro de Teste	RT-RES-013 — Histórico de reservas por hóspede
Objetivo do Roteiro	Verificar se as reservas do hóspede são retornadas em ordem decrescente de criação.
Localização	ReservaService.listarPorHospede(hospedeld) e GET /api/v1/reservas/hospede/{hospedeld}
Pré-condições	Hóspede possui reservas cadastradas em datas diferentes.
Passo a Passo	1. Mockar consulta por hospedeld ordenada. 2. Chamar serviço. 3. Verificar ordenação e mapeamento para DTO.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Branca
Critério de Seleção	Critério de ordenação definido no CDU0019.
Caso de Teste	TC-RES-013 — Listar reservas por hóspede
Entradas	hospedeld=hosp-001.
Resultado Esperado	Lista de ReservaResponse ordenada por dataCriacao desc.
Prioridade	Média
Pós-condições	Histórico retornado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre a consulta de histórico vinculada ao hóspede.

5.14. Listar reservas por hóspede — vazio

Requisito	RF016 / RNF005
Regra de Negócio	RN030
Caso de Uso	CDU0019 — FE01
Roteiro de Teste	RT-RES-014 — Histórico vazio de hóspede
Objetivo do Roteiro	Verificar se hóspede sem reservas retorna lista vazia.
Localização	ReservaService.listarPorHospede(hospedeld)
Pré-condições	Repository retorna lista vazia para o hospedeld.
Passo a Passo	1. Chamar consulta por hóspede sem reservas. 2. Verificar lista vazia. 3. Verificar ausência de

	exceção.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Partição de equivalência
Critério de Seleção	Hóspede com e sem reservas.
Caso de Teste	TC-RES-014 — Listar reservas por hóspede — vazio
Entradas	hospedeld=hosp-sem-reservas.
Resultado Esperado	HTTP 200 com [].
Prioridade	Baixa
Pós-condições	Nenhum dado alterado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante comportamento estável em consultas sem registros.

5.15. Realizar check-in — pagamento no hotel

Requisito	RF017 / RF029
Regra de Negócio	RN018
Caso de Uso	CDU0020 — Fluxo Básico
Roteiro de Teste	RT-RES-015 — Check-in de reserva presencial pendente
Objetivo do Roteiro	Verificar se reserva PENDENTE com pagamento no hotel é alterada para ATIVA.
Localização	ReservaService.realizarCheckIn(id) e POST /api/v1/reservas/{id}/check-in
Pré-condições	Reserva existe; pagamentoModo=PAGAR_NO_HOTEL; status=PENDENTE.
Passo a Passo	1. Buscar reserva. 2. Validar transição. 3. Executar check-in. 4. Verificar status ATIVA, dataHoraCheckIn, histórico e evento.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Branca
Critério de Seleção	Transição válida PENDENTE -> ATIVA para pagamento no hotel.
Caso de Teste	TC-RES-015 — Realizar check-in — pagamento no hotel
Entradas	idReserva=res-balcao-001.
Resultado Esperado	ReservaResponse com status ATIVA e data de check-in preenchida.
Prioridade	Alta
Pós-condições	Reserva ativa e evento de check-in publicado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre a transição principal do fluxo presencial.

5.16. Realizar check-in — pagamento antecipado pago

Requisito	RF017 / RF019 / RF029
Regra de Negócio	RN019 / RN020
Caso de Uso	CDU0020 — Fluxo Básico
Roteiro de Teste	RT-RES-016 — Check-in de reserva antecipada paga
Objetivo do Roteiro	Verificar se reserva antecipada PAGA pode ser ativada.
Localização	ReservaService.realizarCheckIn(id)
Pré-condições	Reserva existe; pagamentoModo=PAGO_ANTECIPADO; reservaStatus=PAGA; pagamentoStatus=PAGO.
Passo a Passo	1. Buscar reserva paga. 2. Executar check-in. 3. Verificar transição para ATIVA. 4. Verificar histórico e evento.
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Transição válida PAGA -> ATIVA para pagamento antecipado.
Caso de Teste	TC-RES-016 — Realizar check-in — pagamento antecipado pago
Entradas	idReserva=res-antecipada-paga.
Resultado Esperado	Status ATIVA; dataHoraCheckIn preenchida; evento de check-in publicado.
Prioridade	Alta
Pós-condições	Reserva antecipada ativa.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante que o pagamento confirmado libera o check-in.

5.17. Realizar check-in — antecipado ainda não pago

Requisito	RF017 / RF019
Regra de Negócio	RN020
Caso de Uso	CDU0020 — FE02
Roteiro de Teste	RT-RES-017 — Bloqueio de check-in sem pagamento confirmado
Objetivo do Roteiro	Verificar se reserva antecipada pendente de pagamento não pode realizar check-in.
Localização	ReservaService.realizarCheckIn(id)
Pré-condições	Reserva existe; pagamentoModo=PAGO_ANTECIPADO; status=PENDENTE; pagamentoStatus=PENDENTE.
Passo a Passo	1. Buscar reserva antecipada pendente. 2. Tentar check-in. 3. Verificar erro de regra de negócio. 4. Verificar que não houve alteração.
Nível de Teste	Unitário
Técnica de Teste	Partição de equivalência
Critério de Seleção	Status de pagamento pendente vs pago.
Caso de Teste	TC-RES-017 — Realizar check-in — antecipado ainda não pago
Entradas	idReserva=res-antecipada-pendente.
Resultado Esperado	Erro de regra de negócio; status permanece PENDENTE.
Prioridade	Alta

Pós-condições	Reserva não alterada.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre regra crítica de bloqueio antes da confirmação de pagamento.

5.18. Realizar check-in — reserva não encontrada ou status incompatível

Requisito	RF017 / RNF001
Regra de Negócio	RN018 / RN019 / RN020
Caso de Uso	CDU0020 — FE01 e FE03
Roteiro de Teste	RT-RES-018 — Check-in inválido
Objetivo do Roteiro	Verificar tratamento para reserva inexistente, encerrada, cancelada ou já ativa quando o check-in não é permitido.
Localização	ReservaService.realizarCheckIn(id)
Pré-condições	Cenários: repository vazio; reserva ENCERRADA; reserva CANCELADA; reserva ATIVA.
Passo a Passo	1. Executar check-in com ID inexistente. 2. Executar com status incompatíveis. 3. Verificar 404 para inexistente e regra de negócio para status inválido.
Nível de Teste	Unitário
Técnica de Teste	Partição de equivalência
Critério de Seleção	Classes de equivalência de status elegíveis e não elegíveis.
Caso de Teste	TC-RES-018 — Realizar check-in — reserva não encontrada ou status incompatível
Entradas	id inexistente; status ENCERRADA/CANCELADA/ATIVA.
Resultado Esperado	Erro 404 ou erro de regra de negócio conforme cenário; nenhuma alteração indevida.
Prioridade	Alta
Pós-condições	Nenhuma transição inválida realizada.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Agrupa cenários negativos de elegibilidade do check-in.

5.19. Realizar check-out — sucesso

Requisito	RF018 / RF028 / RF029
Regra de Negócio	RN021
Caso de Uso	CDU0021 — Fluxo Básico
Roteiro de Teste	RT-RES-019 — Encerramento de reserva ativa
Objetivo do Roteiro	Verificar se reserva ATIVA é encerrada com data de check-out, histórico e evento.
Localização	ReservaService.realizarCheckOut(id) e POST /api/v1/reservas/{id}/check-out
Pré-condições	Reserva existe e está ATIVA.
Passo a Passo	1. Buscar reserva ativa. 2. Executar check-out. 3. Verificar status ENCERRADA, dataHoraCheckOut, histórico e evento. 4. Se houver quartoServicoid, verificar chamada ao serviço de quartos.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Branca
Critério de Seleção	Transição válida ATIVA -> ENCERRADA.
Caso de Teste	TC-RES-019 — Realizar check-out — sucesso
Entradas	idReserva=res-ativa-001.
Resultado Esperado	ReservaResponse com status ENCERRADA; check-out registrado; evento publicado.
Prioridade	Alta
Pós-condições	Reserva encerrada.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre o fluxo principal de finalização da hospedagem.

5.20. Realizar check-out — reserva não ativa ou inexistente

Requisito	RF018 / RNF001
Regra de Negócio	RN021
Caso de Uso	CDU0021 — FE01 e FE02
Roteiro de Teste	RT-RES-020 — Check-out inválido
Objetivo do Roteiro	Verificar se check-out é bloqueado para reserva inexistente ou com status diferente de ATIVA.
Localização	ReservaService.realizarCheckOut(id)
Pré-condições	Cenários com Optional.empty() e reservas PENDENTE/PAGA/ENCERRADA/CANCELADA.
Passo a Passo	1. Tentar check-out com ID inexistente. 2. Tentar com status não ativos. 3. Verificar erros correspondentes e ausência de alteração.
Nível de Teste	Unitário
Técnica de Teste	Partição de equivalência
Critério de Seleção	Status ATIVA vs status não elegíveis.
Caso de Teste	TC-RES-020 — Realizar check-out — reserva não ativa ou inexistente
Entradas	id inexistente; status PENDENTE/PAGA/ENCERRADA/CANCELADA.
Resultado Esperado	Erro 404 para inexistente; erro de regra para status inválido.
Prioridade	Alta
Pós-condições	Reserva não alterada.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante que apenas estadias iniciadas possam ser encerradas.

5.21. Realizar check-out — falha no serviço de quartos não bloqueante

Requisito	RF018 / RF028 / RNF002 / RNF006
------------------	---------------------------------

Regra de Negócio	RN021
Caso de Uso	CDU0021 — FE03
Roteiro de Teste	RT-RES-021 — Resiliência no check-out externo
Objetivo do Roteiro	Verificar se falha ao comunicar check-out ao serviço de quartos é registrada em log e não impede retorno da reserva encerrada.
Localização	ReservaService.realizarCheckOut(id) com QuartoIntegrationClient
Pré-condições	Reserva ATIVA com quartoServicoid; integração externa lança exceção.
Passo a Passo	1. Mockar cliente de quartos para lançar exceção. 2. Executar check-out. 3. Verificar status ENCERRADA. 4. Verificar log ou tratamento da exceção.
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Caminho de exceção não bloqueante da integração.
Caso de Teste	TC-RES-021 — Realizar check-out — falha no serviço de quartos não bloqueante
Entradas	quartoServicoid=Q-101; cliente externo indisponível.
Resultado Esperado	Reserva encerrada retornada; falha registrada; exceção não propagada ao usuário.
Prioridade	Alta
Pós-condições	Reserva encerrada mesmo com falha externa.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre requisito de tolerância a falhas na integração de saída.

5.22. Confirmar pagamento antecipado manualmente — sucesso

Requisito	RF019 / RF029
Regra de Negócio	RN019 / RN020
Caso de Uso	CDU0022 — Fluxo Básico
Roteiro de Teste	RT-RES-022 — Confirmação manual de pagamento
Objetivo do Roteiro	Verificar se reserva antecipada pendente passa para PAGA e pagamentoStatus PAGO.
Localização	ReservaService.confirmarPagamento(id) e POST /api/v1/reservas/{id}/confirma-pagamento
Pré-condições	Reserva existe; pagamentoModo=PAGO_ANTECIPADO; status=PENDENTE; pagamentoStatus=PENDENTE.
Passo a Passo	1. Buscar reserva elegível. 2. Chamar confirmação. 3. Verificar reservaStatus=PAGA e pagamentoStatus=PAGO. 4. Verificar histórico e evento de pagamento confirmado.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo básico do CDU0022.
Caso de Teste	TC-RES-022 — Confirmar pagamento antecipado manualmente — sucesso
Entradas	idReserva=res-antecipada-pendente.
Resultado Esperado	ReservaResponse com status PAGA e pagamentoStatus PAGO.
Prioridade	Alta
Pós-condições	Pagamento antecipado confirmado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre confirmação manual usada por administrador ou funcionário.

5.23. Confirmar pagamento antecipado manualmente — cenários inválidos/idempotência

Requisito	RF019 / RNF001
Regra de Negócio	RN019 / RN020 / RN025
Caso de Uso	CDU0022 — FE01, FE02 e FE03
Roteiro de Teste	RT-RES-023 — Validações da confirmação manual
Objetivo do Roteiro	Verificar tratamento para reserva inexistente, modalidade não antecipada, status não pendente e reserva já paga.
Localização	ReservaService.confirmarPagamento(id)
Pré-condições	Cenários: Optional.empty(); pagamentoModo=PAGAR_NO_HOTEL; reserva CANCELADA/ATIVA/ENCERRADA; reserva já PAGA.
Passo a Passo	1. Executar confirmação em cada cenário. 2. Verificar 404 quando inexistente. 3. Verificar erro de regra quando não antecipada ou não pendente. 4. Verificar retorno sem nova alteração quando já paga.
Nível de Teste	Unitário
Técnica de Teste	Partição de equivalência
Critério de Seleção	Elegibilidade de confirmação e idempotência para já paga.
Caso de Teste	TC-RES-023 — Confirmar pagamento antecipado manualmente — cenários inválidos/idempotência
Entradas	IDs com cenários inválidos.
Resultado Esperado	Erros correspondentes; para já paga, retorno sem duplicar histórico/evento.
Prioridade	Alta
Pós-condições	Nenhuma alteração indevida.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre regras negativas e comportamento idempotente da confirmação manual.

5.24. Registrar comentário de encerramento — sucesso

Requisito	RF020
Regra de Negócio	RN022
Caso de Uso	CDU0023 — Fluxo Básico
Roteiro de Teste	RT-RES-024 — Comentário em reserva encerrada
Objetivo do Roteiro	Verificar se comentário válido é salvo para reserva ENCERRADA.
Localização	ReservaService.registrarComentario(id, request) e POST /api/v1/reservas/{id}/comentarios

Pré-condições	Reserva existe e está ENCERRADA; comentário com texto válido.
Passo a Passo	1. Buscar reserva encerrada. 2. Enviar comentário. 3. Verificar persistência do comentário e DTO retornado.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Preta
Critério de Seleção	Fluxo básico de comentário de encerramento.
Caso de Teste	TC-RES-024 — Registrar comentário de encerramento — sucesso
Entradas	idReserva=res-encerrada; comentario="Hóspede relatou boa estadia".
Resultado Esperado	ReservaComentarioResponse com ID, comentário e vínculo com a reserva.
Prioridade	Média
Pós-condições	Comentário registrado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante registro de informação pós-estadia.

5.25. Registrar comentário — reserva inválida ou texto inválido

Requisito	RF020 / RNF001
Regra de Negócio	RN022
Caso de Uso	CDU0023 — FE01, FE02 e FE03
Roteiro de Teste	RT-RES-025 — Validações do comentário de encerramento
Objetivo do Roteiro	Verificar se o comentário é bloqueado para reserva inexistente, não encerrada, vazio ou maior que 1000 caracteres.
Localização	ReservaService.registrarComentario(id, request) e ReservaController
Pré-condições	Cenários negativos preparados.
Passo a Passo	1. Enviar comentário para ID inexistente. 2. Enviar para reserva ATIVA/PENDENTE. 3. Enviar texto vazio. 4. Enviar texto com 1001 caracteres. 5. Verificar erros.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Partição de equivalência e valor limite
Critério de Seleção	Tamanho mínimo e máximo do comentário e status da reserva.
Caso de Teste	TC-RES-025 — Registrar comentário — reserva inválida ou texto inválido
Entradas	comentario=""; comentario com 1001 caracteres; status não ENCERRADA.
Resultado Esperado	404 para inexistente; erro de regra para status; HTTP 400 para validação de texto.
Prioridade	Média
Pós-condições	Comentário não salvo.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre as principais restrições do registro de comentários.

5.26. Evento externo APROVADO — confirma pagamento

Requisito	RF021 / RF029 / RNF003
Regra de Negócio	RN023
Caso de Uso	CDU0024 — Fluxo Básico
Roteiro de Teste	RT-RES-026 — Consumo de pagamento aprovado
Objetivo do Roteiro	Verificar se evento APROVADO confirma reserva antecipada elegível.
Localização	PagamentoEventConsumer e ReservaService.confirmarPagamentoPorEvento(payload)
Pré-condições	Fila configurada; payload contém reservald e status APROVADO; reserva antecipada pendente existe.
Passo a Passo	1. Simular recebimento do payload. 2. Localizar reserva. 3. Atualizar para PAGA/PAGO. 4. Verificar histórico.
Nível de Teste	Unitário de Consumer / Integração Assíncrona
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo assíncrono de confirmação por mensageria.
Caso de Teste	TC-RES-026 — Evento externo APROVADO — confirma pagamento
Entradas	PagamentoProcessadoPayload{reservald=res-001, status=APROVADO}.
Resultado Esperado	Reserva atualizada para PAGA; pagamentoStatus=PAGO; histórico registrado.
Prioridade	Alta
Pós-condições	Pagamento confirmado por evento externo.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre integração entre serviço de pagamento e reserva.

5.27. Evento externo APROVADO — ignorar ou logar cenários inelegíveis

Requisito	RF021 / RNF002 / RNF003
Regra de Negócio	RN023 / RN025
Caso de Uso	CDU0024 — FE01, FE02 e FE03
Roteiro de Teste	RT-RES-027 — Tratamento de evento aprovado inelegível
Objetivo do Roteiro	Verificar se eventos aprovados para reserva inexistente, não antecipada ou em status final são ignorados/logados sem quebrar o consumer.
Localização	PagamentoEventConsumer
Pré-condições	Cenários: reserva inexistente; pagamento no hotel; reserva PAGA/ATIVA/ENCERRADA/CANCELADA.
Passo a Passo	1. Enviar payload APROVADO para cada cenário. 2. Verificar que não há alteração indevida. 3. Verificar captura de exceção/log quando aplicável.
Nível de Teste	Unitário de Consumer
Técnica de Teste	Partição de equivalência
Critério de Seleção	Elegibilidade de processamento assíncrono.

Caso de Teste	TC-RES-027 — Evento externo APROVADO — ignorar ou logar cenários inelegíveis
Entradas	payloads com status APROVADO e reservad de cenários inelegíveis.
Resultado Esperado	Consumer não propaga exceção; evento ignorado ou logado; status preservado.
Prioridade	Alta
Pós-condições	Nenhuma atualização indevida.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante robustez e idempotência do consumo de eventos.

5.28. Evento externo EXPIRADO — cancela reserva

Requisito	RF022 / RF029 / RNF003
Regra de Negócio	RN024
Caso de Uso	CDU0025 — Fluxo Básico
Roteiro de Teste	RT-RES-028 — Cancelamento por expiração de pagamento
Objetivo do Roteiro	Verificar se evento EXPIRADO cancela reserva antecipada pendente.
Localização	PagamentoEventConsumer e ReservaService.cancelarPorPagamentoExpirado(payload)
Pré-condições	Payload EXPIRADO; reserva antecipada pendente existe.
Passo a Passo	1. Simular recebimento do evento EXPIRADO. 2. Localizar reserva. 3. Alterar reservaStatus para CANCELADA e pagamentoStatus para EXPIRADO. 4. Verificar histórico.
Nível de Teste	Unitário de Consumer / Integração Assíncrona
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo assíncrono de expiração do pagamento.
Caso de Teste	TC-RES-028 — Evento externo EXPIRADO — cancela reserva
Entradas	PagamentoProcessadoPayload{reservad=res-001, status=EXPIRADO}.
Resultado Esperado	Reserva CANCELADA; pagamentoStatus EXPIRADO; histórico registrado.
Prioridade	Alta
Pós-condições	Reserva cancelada por expiração.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre cancelamento automático por falta de pagamento.

5.29. Evento externo EXPIRADO — ignorar ou logar cenários inelegíveis

Requisito	RF022 / RNF002 / RNF003
Regra de Negócio	RN024 / RN025
Caso de Uso	CDU0025 — FE01, FE02 e FE03
Roteiro de Teste	RT-RES-029 — Tratamento de evento expirado inelegível
Objetivo do Roteiro	Verificar se evento EXPIRADO é ignorado quando a reserva não é antecipada, não existe ou já está PAGA/ATIVA/ENCERRADA.
Localização	PagamentoEventConsumer
Pré-condições	Cenários inelegíveis preparados.
Passo a Passo	1. Enviar payload EXPIRADO para reserva inexistente. 2. Enviar para pagamento no hotel. 3. Enviar para status PAGA/ATIVA/ENCERRADA. 4. Verificar logs e ausência de alteração.
Nível de Teste	Unitário de Consumer
Técnica de Teste	Partição de equivalência
Critério de Seleção	Cancelamento permitido vs não permitido.
Caso de Teste	TC-RES-029 — Evento externo EXPIRADO — ignorar ou logar cenários inelegíveis
Entradas	payloads EXPIRADO com diferentes status da reserva.
Resultado Esperado	Consumer não quebra; status preservado; logs gerados quando aplicável.
Prioridade	Alta
Pós-condições	Nenhuma atualização indevida.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante segurança no processamento de eventos de expiração.

5.30. Cadastrar hóspede — sucesso

Requisito	RF023
Regra de Negócio	RN026 / RN027
Caso de Uso	CDU0026 — Fluxo Básico
Roteiro de Teste	RT-HOSP-030 — Cadastro de hóspede válido
Objetivo do Roteiro	Verificar se hóspede válido é salvo como ativo com CPF normalizado.
Localização	HospedeService.criarHospede(request) e POST /api/v1/hospedes
Pré-condições	CPF válido e inexistente.
Passo a Passo	1. Montar request válido. 2. Normalizar CPF. 3. Verificar ausência de duplicidade. 4. Salvar hóspede. 5. Verificar DTO retornado com ativo=true.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Branca e Caixa Preta
Critério de Seleção	Fluxo básico do CDU0026.
Caso de Teste	TC-HOSP-030 — Cadastrar hóspede — sucesso
Entradas	nomeCompleto, cpf=123.456.789-09, email válido, telefone válido.
Resultado Esperado	HospedeResponse com CPF normalizado e ativo=true.
Prioridade	Alta
Pós-condições	Hóspede cadastrado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre o cadastro principal de hóspede.

5.31. Cadastrar hóspede — CPF inválido

Requisito	RF023 / RNF007
Regra de Negócio	RN026
Caso de Uso	CDU0026 — FE01
Roteiro de Teste	RT-HOSP-031 — Validação de CPF inválido
Objetivo do Roteiro	Verificar se CPF inválido bloqueia o cadastro.
Localização	HospedeService.criarHospede(request)
Pré-condições	Request com CPF inválido.
Passo a Passo	1. Montar request com CPF inválido. 2. Chamar criação. 3. Verificar erro de regra de negócio. 4. Verificar que não houve consulta de duplicidade/persistência, se aplicável.
Nível de Teste	Unitário
Técnica de Teste	Partição de equivalência
Critério de Seleção	CPF válido vs inválido.
Caso de Teste	TC-HOSP-031 — Cadastrar hóspede — CPF inválido
Entradas	cpf=111.111.111-11 ou formato inválido.
Resultado Esperado	Erro de regra de negócio; hóspede não salvo.
Prioridade	Alta
Pós-condições	Nenhum hóspede criado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante qualidade do identificador principal do hóspede.

5.32. Cadastrar hóspede — CPF duplicado

Requisito	RF023 / RNF007
Regra de Negócio	RN026
Caso de Uso	CDU0026 — FE02
Roteiro de Teste	RT-HOSP-032 — Bloqueio de duplicidade de CPF
Objetivo do Roteiro	Verificar se CPF já cadastrado retorna erro de conflito.
Localização	HospedeService.criarHospede(request)
Pré-condições	Repository indica existência de hóspede com CPF normalizado.
Passo a Passo	1. Normalizar CPF de entrada. 2. Mockar duplicidade. 3. Chamar criação. 4. Verificar erro de conflito.
Nível de Teste	Unitário
Técnica de Teste	Partição de equivalência
Critério de Seleção	CPF novo vs CPF já existente.
Caso de Teste	TC-HOSP-032 — Cadastrar hóspede — CPF duplicado
Entradas	cpf=123.456.789-09 já cadastrado.
Resultado Esperado	Erro de conflito; hóspede não salvo.
Prioridade	Alta
Pós-condições	Nenhum cadastro duplicado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Evita duplicidade lógica mesmo com máscara diferente no CPF.

5.33. Cadastrar hóspede — campos obrigatórios inválidos

Requisito	RF023 / RNF001
Regra de Negócio	RN026 / RN027
Caso de Uso	CDU0026 — FE03
Roteiro de Teste	RT-HOSP-033 — Validação do request de hóspede
Objetivo do Roteiro	Verificar erro de validação para nome, CPF, e-mail ou telefone inválidos.
Localização	HospedeController POST /api/v1/hospedes
Pré-condições	Validações do DTO habilitadas.
Passo a Passo	1. Enviar JSON com nome vazio. 2. Enviar e-mail inválido. 3. Enviar telefone vazio/inválido. 4. Verificar HTTP 400.
Nível de Teste	Integração HTTP
Técnica de Teste	Caixa Preta
Critério de Seleção	Contrato de entrada do endpoint.
Caso de Teste	TC-HOSP-033 — Cadastrar hóspede — campos obrigatórios inválidos
Entradas	JSON inválido conforme campos obrigatórios.
Resultado Esperado	HTTP 400 Bad Request com mensagens de validação.
Prioridade	Alta
Pós-condições	Hóspede não criado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre validações formais do cadastro.

5.34. Listar hóspedes — com registros

Requisito	RF024 / RNF005
Regra de Negócio	RN030
Caso de Uso	CDU0027 — Fluxo Básico
Roteiro de Teste	RT-HOSP-034 — Listagem de hóspedes
Objetivo do Roteiro	Verificar se todos os hóspedes são retornados em formato DTO.
Localização	HospedeService.listarTodos() e GET /api/v1/hospedes

Pré-condições	Repositório contém hóspedes cadastrados.
Passo a Passo	1. Mockar findAll() com dois hóspedes. 2. Chamar listagem. 3. Verificar tamanho da lista e campos retornados.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Preta
Critério de Seleção	Fluxo básico de consulta.
Caso de Teste	TC-HOSP-034 — Listar hóspedes — com registros
Entradas	Sem parâmetros.
Resultado Esperado	Lista de HospedeResponse com os registros existentes.
Prioridade	Média
Pós-condições	Lista retornada.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante consulta geral de hóspedes.

5.35. Listar hóspedes — lista vazia

Requisito	RF024 / RNF005
Regra de Negócio	RN030
Caso de Uso	CDU0027 — FE01
Roteiro de Teste	RT-HOSP-035 — Listagem vazia de hóspedes
Objetivo do Roteiro	Verificar se ausência de hóspedes retorna lista vazia.
Localização	HospedeService.listarTodos() e GET /api/v1/hospedes
Pré-condições	Repository retorna lista vazia.
Passo a Passo	1. Chamar endpoint/serviço. 2. Verificar HTTP 200 e array vazio.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Partição de equivalência
Critério de Seleção	Consulta com e sem registros.
Caso de Teste	TC-HOSP-035 — Listar hóspedes — lista vazia
Entradas	Sem parâmetros.
Resultado Esperado	HTTP 200 com [].
Prioridade	Baixa
Pós-condições	Nenhum dado alterado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre regra de retorno vazio sem erro.

5.36. Buscar hóspede por ID — sucesso

Requisito	RF025
Regra de Negócio	RN030
Caso de Uso	CDU0028 — Fluxo Básico
Roteiro de Teste	RT-HOSP-036 — Busca de hóspede por ID
Objetivo do Roteiro	Verificar se hóspede existente é retornado pelo ID.
Localização	HospedeService.buscarPorId(id) e GET /api/v1/hospedes/{id}
Pré-condições	Hóspede existente.
Passo a Passo	1. Mockar findById. 2. Chamar busca. 3. Verificar DTO e HTTP 200.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Preta
Critério de Seleção	Fluxo básico de busca por identificador.
Caso de Teste	TC-HOSP-036 — Buscar hóspede por ID — sucesso
Entradas	id=hosp-001.
Resultado Esperado	HospedeResponse correspondente ao ID informado.
Prioridade	Média
Pós-condições	Hóspede retornado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Valida consulta individual de hóspede.

5.37. Buscar hóspede por ID — não encontrado

Requisito	RF025 / RNF001
Regra de Negócio	RN030
Caso de Uso	CDU0028 — FE01
Roteiro de Teste	RT-HOSP-037 — Hóspede inexistente por ID
Objetivo do Roteiro	Verificar se ID inexistente retorna recurso não encontrado.
Localização	HospedeService.buscarPorId(id)
Pré-condições	Repository retorna Optional.empty().
Passo a Passo	1. Chamar busca com ID inexistente. 2. Verificar erro 404. 3. Verificar que não há efeitos colaterais.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Partição de equivalência
Critério de Seleção	ID existente vs inexistente.
Caso de Teste	TC-HOSP-037 — Buscar hóspede por ID — não encontrado
Entradas	id=inexistente.
Resultado Esperado	Erro 404/recurso não encontrado.
Prioridade	Média
Pós-condições	Nenhum dado alterado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.

Classificação	Planejado
Descrição do Resultado	Cobre cenário negativo da busca por ID.

5.38. Buscar hóspede por CPF — sucesso

Requisito	RF025 / RNF007
Regra de Negócio	RN026
Caso de Uso	CDU0029 — Fluxo Básico
Roteiro de Teste	RT-HOSP-038 — Busca de hóspede por CPF normalizado
Objetivo do Roteiro	Verificar se CPF válido é normalizado e usado para buscar o hóspede.
Localização	HospedeService.buscarPorCpf(cpf) e GET /api/v1/hospedes/cpf/{cpf}
Pré-condições	Hóspede cadastrado com CPF normalizado.
Passo a Passo	1. Informar CPF com máscara. 2. Normalizar CPF. 3. Buscar hóspede. 4. Verificar DTO retornado.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Branca
Critério de Seleção	Normalização antes da consulta.
Caso de Teste	TC-HOSP-038 — Buscar hóspede por CPF — sucesso
Entradas	cpf=123.456.789-09.
Resultado Esperado	HospedeResponse correspondente ao CPF 12345678909.
Prioridade	Média
Pós-condições	Hóspede retornado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre busca por documento com máscara de entrada.

5.39. Buscar hóspede por CPF — CPF inválido ou inexistente

Requisito	RF025 / RNF001 / RNF007
Regra de Negócio	RN026
Caso de Uso	CDU0029 — FE01 e FE02
Roteiro de Teste	RT-HOSP-039 — Validações da busca por CPF
Objetivo do Roteiro	Verificar se CPF inválido retorna erro de regra e CPF válido inexistente retorna 404.
Localização	HospedeService.buscarPorCpf(cpf)
Pré-condições	Cenários com CPF inválido e CPF válido não cadastrado.
Passo a Passo	1. Buscar com CPF inválido. 2. Verificar erro de regra. 3. Buscar com CPF válido inexistente. 4. Verificar 404.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Partição de equivalência
Critério de Seleção	CPF inválido, válido inexistente e válido existente.
Caso de Teste	TC-HOSP-039 — Buscar hóspede por CPF — CPF inválido ou inexistente
Entradas	cpf inválido; cpf=987.654.321-00 não cadastrado.
Resultado Esperado	Erro de regra para CPF inválido; 404 para CPF inexistente.
Prioridade	Média
Pós-condições	Nenhum dado alterado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre validações do endpoint de busca por CPF.

5.40. Atualizar hóspede — sucesso

Requisito	RF026 / RNF007
Regra de Negócio	RN026 / RN028
Caso de Uso	CDU0030 — Fluxo Básico
Roteiro de Teste	RT-HOSP-040 — Atualização de cadastro de hóspede
Objetivo do Roteiro	Verificar se dados e status ativo são atualizados quando o hóspede existe e o CPF é válido.
Localização	HospedeService.atualizarHospede(id, request) e PUT /api/v1/hospedes/{id}
Pré-condições	Hóspede existente; novo CPF válido e não usado por outro hóspede.
Passo a Passo	1. Buscar hóspede existente. 2. Validar e normalizar CPF. 3. Verificar duplicidade em outro hóspede. 4. Atualizar campos. 5. Verificar DTO retornado.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo básico do CDU0030.
Caso de Teste	TC-HOSP-040 — Atualizar hóspede — sucesso
Entradas	id=hosp-001; body com nome, cpf, email, telefone, ativo=false.
Resultado Esperado	HospedeResponse com dados atualizados e status ativo conforme request.
Prioridade	Alta
Pós-condições	Dados do hóspede atualizados.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre manutenção cadastral do hóspede.

5.41. Atualizar hóspede — cenários inválidos

Requisito	RF026 / RNF001 / RNF007
Regra de Negócio	RN026 / RN028
Caso de Uso	CDU0030 — FE01, FE02, FE03 e FE04
Roteiro de Teste	RT-HOSP-041 — Validações da atualização de hóspede

Objetivo do Roteiro	Verificar tratamento para hóspede inexistente, CPF inválido, CPF de outro hóspede e campos inválidos.
Localização	HospedeService.atualizarHospede(id, request) e HospedeController
Pré-condições	Cenários negativos preparados.
Passo a Passo	1. Atualizar ID inexistente. 2. Atualizar com CPF inválido. 3. Atualizar com CPF pertencente a outro hóspede. 4. Enviar campos obrigatórios inválidos. 5. Verificar erros.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Partição de equivalência
Critério de Seleção	Validação de existência, CPF e contrato de entrada.
Caso de Teste	TC-HOSP-041 — Atualizar hóspede — cenários inválidos
Entradas	id inexistente; cpf inválido; cpf de outro registro; body inválido.
Resultado Esperado	404, erro de regra, conflito ou HTTP 400 conforme cenário.
Prioridade	Alta
Pós-condições	Nenhuma atualização indevida.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Agrupar cenários negativos da atualização cadastral.

5.42. Excluir hóspede — sucesso

Requisito	RF027
Regra de Negócio	RN029
Caso de Uso	CDU0031 — Fluxo Básico
Roteiro de Teste	RT-HOSP-042 — Exclusão de hóspede sem reservas
Objetivo do Roteiro	Verificar se hóspede existente sem reservas vinculadas é removido.
Localização	HospedeService.excluirHospede(id) e DELETE /api/v1/hospedes/{id}
Pré-condições	Hóspede existe e não possui reservas vinculadas.
Passo a Passo	1. Buscar hóspede. 2. Verificar inexistência de reservas vinculadas. 3. Executar exclusão. 4. Verificar HTTP 204.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo básico do CDU0031.
Caso de Teste	TC-HOSP-042 — Excluir hóspede — sucesso
Entradas	id=hosp-sem-reservas.
Resultado Esperado	delete() chamado; resposta HTTP 204 sem corpo.
Prioridade	Alta
Pós-condições	Hóspede removido do sistema.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobrir exclusão permitida de hóspede sem dependências.

5.43. Excluir hóspede — inexistente ou com reservas vinculadas

Requisito	RF027 / RNF001
Regra de Negócio	RN029
Caso de Uso	CDU0031 — FE01 e FE02
Roteiro de Teste	RT-HOSP-043 — Bloqueios de exclusão de hóspede
Objetivo do Roteiro	Verificar se exclusão é bloqueada para hóspede inexistente ou com reservas vinculadas.
Localização	HospedeService.excluirHospede(id)
Pré-condições	Cenários: Optional.empty(); hóspede com count de reservas maior que zero.
Passo a Passo	1. Excluir ID inexistente e verificar 404. 2. Excluir hóspede com reservas e verificar conflito. 3. Verificar que delete() não é chamado.
Nível de Teste	Unitário / Integração HTTP
Técnica de Teste	Partição de equivalência
Critério de Seleção	Exclusão permitida vs bloqueada por dependência.
Caso de Teste	TC-HOSP-043 — Excluir hóspede — inexistente ou com reservas vinculadas
Entradas	id inexistente; id=hosp-com-reservas.
Resultado Esperado	404 para inexistente; 409 Conflict para reservas vinculadas.
Prioridade	Alta
Pós-condições	Hóspede preservado.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Garante integridade referencial do cadastro de hóspedes.

5.44. Autorização por atores permitidos

Requisito	RNF004
Regra de Negócio	RN012 a RN030
Caso de Uso	CDU0016 a CDU0031
Roteiro de Teste	RT-NF-044 — Controle de acesso dos endpoints
Objetivo do Roteiro	Verificar se apenas administrador e funcionário acessam os endpoints de reservas e hóspedes.
Localização	ReservaController e HospedeController com camada de segurança habilitada
Pré-condições	Usuários simulados com perfis ADMINISTRADOR, FUNCIONARIO e perfil não autorizado.
Passo a Passo	1. Executar requisições autenticadas como administrador. 2. Executar como funcionário. 3. Executar como usuário sem permissão. 4. Verificar status HTTP.
Nível de Teste	Integração HTTP / Segurança
Técnica de Teste	Caixa Preta
Critério de Seleção	Controle de acesso por perfil.
Caso de Teste	TC-NF-044 — Autorização por atores permitidos

Entradas	Tokens/usuários simulados por perfil.
Resultado Esperado	Administrador e funcionário recebem 2xx/4xx funcionais; perfil não autorizado recebe 403.
Prioridade	Alta
Pós-condições	Acesso restrito aos atores definidos.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre requisito não funcional de segurança quando a autenticação estiver habilitada.

5.45. Padronização de erros HTTP

Requisito	RNF001
Regra de Negócio	RN012 a RN030
Caso de Uso	CDU0016 a CDU0031
Roteiro de Teste	RT-NF-045 — Contrato de erro padronizado
Objetivo do Roteiro	Verificar se erros de validação, regra, conflito, integração e não encontrado retornam estrutura padronizada.
Localização	ExceptionHandler global / Controllers
Pré-condições	Cenários de erro simulados para cada categoria.
Passo a Passo	1. Forçar erro de validação. 2. Forçar regra de negócio. 3. Forçar conflito. 4. Forçar integração. 5. Forçar recurso não encontrado. 6. Verificar código e corpo.
Nível de Teste	Integração HTTP
Técnica de Teste	Caixa Preta
Critério de Seleção	Categorias de erro expostas pela API.
Caso de Teste	TC-NF-045 — Padronização de erros HTTP
Entradas	Requests inválidos e mocks que lançam exceções específicas.
Resultado Esperado	Respostas com status HTTP coerente, timestamp/mensagem/código de erro quando definido.
Prioridade	Alta
Pós-condições	Contrato de erro consistente.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Facilita consumo da API e diagnóstico por outros serviços.

5.46. Logs em falhas assíncronas e integrações

Requisito	RNF002 / RNF003 / RNF006
Regra de Negócio	RN021 / RN025
Caso de Uso	CDU0021, CDU0024 e CDU0025
Roteiro de Teste	RT-NF-046 — Observabilidade de falhas controladas
Objetivo do Roteiro	Verificar se falhas esperadas em integração de quartos e consumers são registradas em log sem interromper o fluxo quando a regra assim define.
Localização	ReservaService, QuartoIntegrationClient e PagamentoEventConsumer
Pré-condições	Mocks configurados para lançar exceções controladas.
Passo a Passo	1. Simular falha no check-out externo. 2. Simular evento para reserva inexistente. 3. Verificar captura da exceção e registro de log com reservald/status.
Nível de Teste	Unitário / Integração Assíncrona
Técnica de Teste	Caixa Branca
Critério de Seleção	Caminhos de exceção controlados.
Caso de Teste	TC-NF-046 — Logs em falhas assíncronas e integrações
Entradas	Exceções de cliente externo e payloads inválidos.
Resultado Esperado	Erro registrado em log; consumer/serviço não propaga exceção indevida.
Prioridade	Média
Pós-condições	Falha diagnosticável sem derrubar fluxo.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre requisito de observabilidade e resiliência.

5.47. Desempenho básico de consultas

Requisito	RNF005
Regra de Negócio	RN030
Caso de Uso	CDU0017, CDU0019 e CDU0027
Roteiro de Teste	RT-NF-047 — Tempo de resposta das listagens
Objetivo do Roteiro	Verificar se listagens e buscas simples respondem em tempo aceitável no ambiente de teste.
Localização	Endpoints GET de reservas e hóspedes
Pré-condições	Base de teste com volume controlado de registros.
Passo a Passo	1. Popular base com volume definido. 2. Executar GET /reservas, GET /hospedes e GET /reservas/hospede/{id}. 3. Medir tempo de resposta. 4. Comparar com limite definido pelo projeto.
Nível de Teste	Sistema / Integração
Técnica de Teste	Teste de desempenho básico
Critério de Seleção	Caminhos de consulta mais usados.
Caso de Teste	TC-NF-047 — Desempenho básico de consultas
Entradas	Base com 100 a 1000 registros de teste.
Resultado Esperado	Tempo de resposta dentro do limite acordado; sem erro 5xx.
Prioridade	Média
Pós-condições	Consultas respondem de forma estável.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre requisito não funcional de estabilidade mínima em consultas.

5.48. Rastreabilidade por histórico e eventos

Requisito	RF029 / RNF008
Regra de Negócio	RN016 / RN017 / RN021 / RN023 / RN024
Caso de Uso	CDU0016, CDU0020, CDU0021, CDU0022, CDU0024 e CDU0025
Roteiro de Teste	RT-NF-048 — Auditoria de alterações de reserva
Objetivo do Roteiro	Verificar se operações que alteram estado de reserva registram histórico e publicam eventos quando previsto.
Localização	ReservaService e publishers de eventos
Pré-condições	Mocks de repositório de histórico e publisher configurados.
Passo a Passo	1. Criar reserva. 2. Confirmar pagamento. 3. Realizar check-in. 4. Realizar check-out. 5. Processar eventos APROVADO/EXPIRADO. 6. Verificar histórico e publicação esperada em cada operação.
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Pontos de mudança de estado do domínio.
Caso de Teste	TC-NF-048 — Rastreabilidade por histórico e eventos
Entradas	Operações de alteração de status em reserva.
Resultado Esperado	Histórico salvo com status anterior/novo e eventos publicados conforme modalidade e fluxo.
Prioridade	Alta
Pós-condições	Rastreabilidade mantida.
Resultado Obtido	A executar — caso de teste planejado, sem evidência de execução informada.
Classificação	Planejado
Descrição do Resultado	Cobre requisito transversal de auditoria do fluxo de reservas.

6. RESUMO DA CLASSIFICAÇÃO DOS RESULTADOS OBTIDOS

Nível de Testes	#Total Planejado	#Executados	#Validados	#Pendentes	#Bugs (urg., alto, médio, baixo)	#Melhorias (necess., desej.)
Testes Unitários (Service/Domain)	30	0	0	30	0, 0, 0, 0	0, 0
Testes de Integração HTTP (Controller/Web)	10	0	0	10	0, 0, 0, 0	0, 0
Testes de Consumer/Mensageria	5	0	0	5	0, 0, 0, 0	0, 0
Testes Não Funcionais/Transversais	3	0	0	3	0, 0, 0, 0	0, 0
TOTAL	48	0	0	48	0, 0, 0, 0	0, 0

Observação: os totais acima classificam os casos conforme o foco principal do teste. Alguns casos possuem natureza híbrida, por exemplo unitário e integração HTTP, mas foram contabilizados apenas uma vez para evitar duplicidade.

7. CONCLUSÃO

reserva-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
br.edu.udesc.reservaservice.application.service		62%		60%	47	74	64	192	22	40	2	4
br.edu.udesc.reservaservice.api.controller		20%		0%	21	23	38	54	19	21	1	3
br.edu.udesc.reservaservice.domain.model		61%		50%	36	52	52	122	11	20	2	5
br.edu.udesc.reservaservice.infrastructure.config		23%		n/a	17	21	39	51	17	21	3	7
br.edu.udesc.reservaservice.infrastructure.integration.client		11%		n/a	10	11	36	37	10	11	3	4
br.edu.udesc.reservaservice.infrastructure.integration.gateway		66%		50%	15	29	25	66	13	27	2	6
br.edu.udesc.reservaservice.infrastructure.messaging.producer		60%		28%	8	14	26	66	4	9	3	5
br.edu.udesc.reservaservice.application.mapper		63%		n/a	6	13	30	87	6	13	0	3
br.edu.udesc.reservaservice.api.response		64%		n/a	4	7	4	7	4	7	4	7
br.edu.udesc.reservaservice.application.dto		76%		n/a	3	8	3	8	3	8	3	8
br.edu.udesc.reservaservice.infrastructure.messaging.consumer		59%		75%	4	9	10	26	3	7	1	3
br.edu.udesc.reservaservice.exception		46%		n/a	4	8	9	17	4	8	0	1
br.edu.udesc.reservaservice.api.request		60%		n/a	3	5	3	5	3	5	3	5
br.edu.udesc.reservaservice.infrastructure.integration.fallback		57%		25%	3	6	5	12	1	4	0	2
br.edu.udesc.reservaservice.domain.enums		82%		n/a	1	5	4	23	1	5	1	5
br.edu.udesc.reservaservice.shared.base		18%		n/a	2	3	6	7	2	3	0	1
br.edu.udesc.reservaservice.domain.exception		73%		n/a	2	9	4	18	2	9	1	8
br.edu.udesc.reservaservice.shared.util		82%		66%	2	6	3	8	0	3	0	2
br.edu.udesc.reservaservice		37%		n/a	1	2	2	3	1	2	0	1
br.edu.udesc.reservaservice.infrastructure.security		0%		n/a	2	2	2	2	2	2	1	1
br.edu.udesc.reservaservice.domain.event		100%		n/a	0	8	0	24	0	8	0	2
br.edu.udesc.reservaservice.infrastructure.messaging.payload		100%		n/a	0	4	0	4	0	4	0	4
Total	1.541 of 3.675	58%	76 of 161	52%	191	319	365	839	128	237	30	87